5. relocateimage() 先重定位 libc、再重定位主镜像：处理 .rela.dyn 与 .rela.plt，符号解析顺序为“主镜像优先、libc 兜底”，未定义的强符号会让 exec() 直接失败。
6. 校验入口合法、布置 argv 栈、把 PC 设为 mainimg.base + e_entry；由于重定位用 copyout 改写了用户代码与 GOT 页，返回用户态前必须 uvmsync_icache() 做 I-cache 维护。

支持的重定位类型. 装载器只认这份 AArch64 libc 实际会生成的四种 R_AARCH64_*，且全部在装载期一次性（eager）绑定，没有延迟 PLT：

```
switch(type){
case R_AARCH64_NONE:                        // 跳过
  continue;
case R_AARCH64_RELATIVE:                    // 基址 + addend
  value = img->base + rela.addend; break;
case R_AARCH64_ABS64:                       // 解析符号后 + addend
case R_AARCH64_GLOB_DAT:
case R_AARCH64_JUMP_SLOT:
  value = resolvesym(...) + rela.addend; break;
default:                                    // 其它类型一律失败
  printf("exec: unsupported AArch64 dynamic relocation %d\n", type);
  return -1;
}
```

真机上以默认配置启动、登录后，串口打印每个动态程序里 libc.so 的载入基址，这两行即证明主 ELF 与 libc.so 都被成功装载、符号被解析、重定位已完成：

```
exec: dynamic /bin/login, libc.so base=0x0000000000002000
xv6-rpi3 login: root
Password: xv6
Welcome root
exec: dynamic /bin/sh, libc.so base=0x0000000000003000
```

这里的 base 就是上文的 libbase：login 主镜像最大到 0x1a20，页对齐后落在第 2 页 0x2000；sh 更大，跨到第 3 页 0x3000。若某个程序没有打印 exec: dynamic 却能运行，说明它是静态 ET_EXEC 回退路径。

收敛与限制. 这是一个刻意为之的最小 **xv6 原生**装载 ABI，而不是完整的 ld.so：只支持 RELA（无 DT_REL）；只认上面四种动态重定位，带 GOT 数据重定位（ADR_GOT_PAGE/LD64_GOT_LO12_NC）的二进制会失败；只允许受信任的单个 /lib/libc.so，libc 自身不得再有依赖；没有 PT_INTERP、dlopen/dlsym、延迟 PLT 绑定、IFUNC、TLS、符号 interposition 与 ASLR；libc.so 每次 exec 都重新读盘并复制进本进程地址空间，进程间不共享只读物理页。于是它消除了每个程序静态携带 libc 的磁盘冗余，却还没省下运行时的物理内存。可插拔 .ko 模块的装载器与它共用 kernel/elf.h 里的数据结构，却是两套独立逻辑：模块装载器处理 ET_REL、做的是指令位域修补，而这里处理 ET_DYN、做的是把基址与符号值写回 64 位槽。

8.5.3 POSIX 层

目标是让只依赖常见 POSIX 接口的 C 源码能直接用 xv6 工具链重新编译。它由三部分组成：

1. include/ 提供标准名字的头文件：stdio.h、stdlib.h、string.h、unistd.h、fcntl.h、errno.h、time.h、pthread.h、semaphore.h、sys/types.h、sys/stat.h、sys/socket.h、netinet/in.h、arpa/inet.h、sys/epoll.h；
2. user/posix.c 把标准 API 映射到 xv6 已有的能力；